

Testēšanas plāna izstrāde

Sasniedzamie rezultāti:

- Sastāda produkta testēšanas plānu.
- Nosaka testēšanas gadījumus.
- Skaidro testēšanas nozīmi izstrādes procesā.

1. uzdevums. Kāpēc nepieciešama testēšana? Atbilde uz jautājumiem.

1.1. Kas ir programmatūras testēšana?

Programmatūras testēšana ir process, kurā pārbauda programmu, lai atrastu kļūdas un pārliecinātos, ka tā darbojas pareizi un atbilst prasībām.

1.2. Kāpēc testēšana jāveic jau izstrādes laikā?

Testēšana jāveic izstrādes laikā, lai laikus atklātu kļūdas, tās būtu vieglāk izlabot un lai gala produkts būtu kvalitatīvāks.

1.3. Nosauc vismaz 3 problēmas, kas var rasties netestētā programmā.

- Programma var nedarboties pareizi
- Var rasties kļūdas lietotāja ievadē
- Lietotāja saskarne var būt neērta vai bojāta
- Funkcijas var nestrādāt kā paredzēts

2. uzdevums. Produkta funkcionalitātes analīze. Nosaki svarīgākās funkcijas savā projektā.

Funkcionalitāte	Lietotāja darbība	Sagaidāmais rezultāts
Uzdevuma pievienošana	Ievada uzdevumu un nospiež pogu	Uzdevums tiek pievienots sarakstam
Uzdevuma dzēšana	Nospiež dzēšanas pogu	Uzdevums tiek izdzēsts
Uzdevuma atzīmēšana kā izpildīts	Atzīmē checkbox	Uzdevums kļūst atzīmēts kā izpildīts
Uzdevumu saglabāšana	Aizver un atver lapu	Uzdevumi saglabājas (ja ir storage)

3. uzdevums. Ko nepieciešams testēt? Nosaki galvenos aspektus, kas jātestē Tavā produktā.

Testējamais aspekts	Kāpēc tas jāpārbauda
Funkcionalitāte	Lai pārliecinātos, ka visas funkcijas darbojas pareizi
Lietotāja ievade	Lai novērstu kļūdas un nepareizu datu ievadi
Dizains un izkārtojums	Lai lietotājam būtu ērti lietot programmu
JavaScript darbība	Lai pārliecinātos, ka interaktīvās funkcijas strādā

4. uzdevums. Testēšanas gadījumu plānošana. Izveido testēšanas gadījumus, kas palīdzēs pārbaudīt Tava produkta kvalitāti izstrādes laikā.

Testa Nr	Ko pārbaudīsi	Lietotāja darbība	Sagaidāmais rezultāts
1	Uzdevuma pievienošana	Ievada tekstu un nospiež “Add”	Uzdevums parādās sarakstā
2	Tukša ievade	Nospiež “Add” bez teksta	Parādās kļūdas paziņojums
3	Uzdevuma dzēšana	Nospiež “Delete”	Uzdevums pazūd
4	Checkbox darbība	Atzīmē uzdevumu	Uzdevums kļūst izpildīts
5	Saglabāšana	Pārlādē lapu	Uzdevumi saglabājas

5. uzdevums. Kļūdainas ievades plānošana. Padomā, kā lietotājs var ievadīt nepareizus datus.

Ievades lauks	Iespējamā kļūdainā ievade	Kā sistēmai jāreaģē
Uzdevuma ievade	Tukšs lauks	Parāda kļūdas paziņojumu un nepieļauj pievienošanu
Uzdevuma ievade	Tikai atstarpes	Automātiski ignorē vai parāda kļūdu
Uzdevuma ievade	Ļoti garš teksts	Ierobežo simbolu skaitu vai korekti attēlo tekstu
Uzdevuma ievade	HTML vai skriptu kods	Neizpilda kodu (drošības nolūkos), attēlo kā tekstu

6. uzdevums. Testēšanas plāna izvērtējums. Atbilde uz jautājumiem.

Kā Tavs testēšanas plāns palīdzēs izveidot kvalitatīvāku produktu?

Tas palīdz laikus atrast kļūdas, uzlabot lietotāja pieredzi un nodrošināt, ka visas funkcijas darbojas pareizi.

Kad izstrādes procesā Tu veiksi testēšanu?

- | | | | | |
|-------------------------------------|-----|----------|-----------|----------|
| ☒ | pēc | katras | funkcijas | izveides |
| ☐ | | projekta | | beigās |

☒ regulāri izstrādes laikā

Pamato savu izvēli:

Testējot regulāri un pēc katras funkcijas izveides, kļūdas var atklāt ātrāk un tās ir vieglāk izlabot, kas uzlabo kopējo produkta kvalitāti.